

BRDS-PEC: Behavioral Ransomware Detection System with Pre-Encryption Containment

Complete 30-Page Technical Architecture, Machine Learning Mechanics & Operational Manual

Module 1: Executive Overview & Project Vision

BRDS-PEC (Behavioral Ransomware Detection System with Pre-Encryption Containment) is an end-to-end, enterprise-ready endpoint security system engineered to detect and neutralize zero-day ransomware executions in the **pre-encryption phase (5 to 15 seconds after execution)**. Rather than relying on static file signatures or SHA-256 file hashes—which malware authors effortlessly bypass using automated re-packing—BRDS-PEC continuously monitors low-level operating system behavioral telemetry captured via Microsoft Sysmon v15+.

The Plain-English Analogy for Non-Technical Stakeholders:

Imagine a high-security bank guarded by an officer who only checks a printed photo album of known bank robbers (Signature Antivirus). If a robber puts on a new mask or changes clothes (polymorphic zero-day ransomware), the signature guard lets them walk right through the front door. BRDS-PEC acts like an intelligent AI behavioral guard standing inside the vault. It does not care what the robber looks like—it watches what they *do*. If a process starts carrying a crowbar, smashing windows, deleting emergency keys (`\vssadmin delete shadows``), and preparing to lock the vault doors, BRDS-PEC tackles the burglar and isolates the room in 5 to 15 seconds—**before the vault doors are locked**.

Module 1 (Continued): System Scope & Core Objectives

Core System Objectives:

1. **Zero Data Loss Guarantee:** Intercept ransomware during process initialization and shadow copy deletion, prior to mass file payload encryption.
2. **Ultra-Low False Positive Rate:** Maintain a False Positive Rate under 0.25% (achieved 0.22%) on routine Windows 11 workloads.
3. **Automated Safe Containment:** Terminate malicious process trees while protecting core OS processes to prevent Blue Screen of Death (BSOD) crashes.

Modern enterprise networks face an unprecedented wave of human-operated and automated ransomware campaigns. To provide a complete solution, BRDS-PEC establishes 4 strict operational boundaries:

- **Boundary 1: Kernel-Level System Event Observability** — Leverages native Windows Event Tracing (ETW) via Sysmon driver hooks rather than unstable user-mode API hooking.
- **Boundary 2: Sub-Minute Detection Window** — Operates on 5-second sliding UTC temporal windows to capture pre-encryption signals before mass file modification.
- **Boundary 3: Cryptographic Anti-Tampering** — Enforces HMAC-SHA256 digests and single-use `.arm_token`` authorization files to prevent alert forging.
- **Boundary 4: Non-Disruptive Host Isolation** — Disables active network adapters (``Disable-NetAdapter``) and collapses malicious PIDs while protecting core Windows system processes (``lsass``, ``csrss``, ``explorer``).

Module 2: The Failure of Traditional Cybersecurity & Threat Landscape

Modern ransomware strains (e.g. **LockBit 3.0**, **WannaCry**, **BlackBasta**, **Ryuk**, **Sodinokibi**) have evolved to exploit the fundamental architectural vulnerabilities of traditional signature-based Endpoint Detection and Response (EDR) platforms.

- 1. Polymorphic Packing & Hash Evasion:** Ransomware authoring kits use polymorphic encryption (e.g. UPX, Themida) to re-compile binary payloads every few minutes. A single bit change produces a completely new SHA-256 hash, rendering static blocklists completely useless.
- 2. Extreme Encryption Velocity:** High-performance ransomware utilizes multi-threaded I/O completion ports and Windows CryptoAPI to encrypt files at speeds exceeding 25,000 files per minute.
- 3. Pre-Encryption Shadow Copy Destruction:** Prior to encrypting files, ransomware executes automated subprocesses (`cmd.exe /c vssadmin delete shadows /all /quiet``, `wmic shadowcopy delete``, `bcdedit /set {default} recoveryenabled No``). This destroys volume shadow restore points, leaving victims zero recovery options.
- 4. Post-Encryption Alerting Flaw:** Legacy EDR tools often fire alerts ONLY after ransom notes (`READ_ME.txt``, `DECRYPT.html``) appear on the user's desktop—meaning detection happens *after* data loss has occurred.

Module 2 (Continued): Comparative Defense Matrix

The following matrix illustrates why traditional security solutions fail and how BRDS-PEC overcomes each limitation:

Vector / Feature	Legacy Antivirus	Standard EDR	BRDS-PEC Architecture
Detection Target	Known SHA-256 File Hashes	Heuristic Rules & YARA	17-Dimensional Sysmon Behaviors
Reaction Time	Hours to Days (Post-Outbreak)	1 to 5 Minutes (Post-Encryption)	5 to 15 Seconds (Pre-Encryption)
Zero-Day Protection	Fails (Requires updated DB)	Partial (Bypassed by packing)	93.14% Held-Out Zero-Day F1 Score
Backup Protection	None	Monitors VSS API after alert	Halts `vssadmin` process tree immediately
Explainability	Static Signature Match String	Rule Name / Alert ID	PyTorch Autograd SHAP XAI Reports

Module 3: Step 1 — Windows Sysmon Telemetry Ingestion Deep Dive

BRDS-PEC relies on Microsoft System Monitor (Sysmon v15+), a Windows system service driver that hooks directly into the Windows kernel to capture low-level operating system activity. Telemetry is ingested in real-time by pipeline/watchdog.py via the Windows Event Channel Microsoft-Windows-Sysmon/Operational.

Sysmon Event ID	Event Name	Kernel Activity & Ransomware Indicator Context
Event ID 1	Process Creation	Captures parent-child process chains (e.g. cmd.exe spawning vssadmin.exe to wipe shadow copies).
Event ID 3	Network Connection	Tracks outbound TCP/UDP socket creation for C2 beaconing or SMB lateral movement.
Event ID 7	Image Loaded / DLL Load	Detects stealth DLL injections into legitimate Windows processes (explorer.exe, svchost.exe).
Event ID 11	File Create / Rename	Detects high-frequency file renaming bursts appending ransomware extensions (.lockbit, .WNCRY).
Event IDs 12 & 13	Registry Create / Value Set	Tracks modifications to Windows startup registry keys (Run keys) and disabling Windows Defender.
Event IDs 23 & 26	File Delete / Delete Detected	Captures mass file wiping operations prior to payload encryption.

Module 3 (Continued): Real-Time Watchdog Ingestion Pipeline

The live watchdog script (pipeline/watchdog.py) runs continuously as a system background service. It utilizes the PyWin32 API (win32evtlog) to query the Microsoft-Windows-Sysmon/Operational log buffer without dropping events.

```
def listen_live_sysmon(): query_handle = win32evtlog.EvtQuery('Microsoft-Windows-Sysmon/Operational',
win32evtlog.EvtQueryChannelPath) while True: events = win32evtlog.EvtNext(query_handle, 100) for event
in events: xml_content = win32evtlog.EvtRender(event, win32evtlog.EvtRenderEventXml) parsed_event =
parse_sysmon_xml(xml_content) aggregator.push(parsed_event)
```

Module 4: Step 2 — Temporal Aggregation & 17 Feature Vectorization

Raw Sysmon event streams are processed by pipeline/temporal_aggregator.py into **5-second sliding UTC windows**. Events are grouped by (computer, process_key, window_start) and converted into a 17-dimensional quantitative feature vector by pipeline/vectorizer.py:

#	Feature Name	Extraction Logic & Formula	Ransomware Behavioral Indicator
1	event_count	Total event count in 5s window	Spikes (>50) during automated attack execution.
2	unique_images	Count of unique binary paths	Spikes (>3) during multi-process spawning.
3	unique_files	Count of unique file targets	Spikes (>20) during mass file targeting.
4	unique_extensions	Count of new file extensions	Spikes (>1) when appending custom extensions.
5	unique_destination_ips	Count of unique remote IPs	Spikes (>2) during C2 beaconing or network scan.
6	suspicious_path_count	Executions from Temp/AppData/Public	High (>=1) for dropped malicious binaries.
7	file_activity_count	Sum of Event IDs 11, 23, 26	Massive spike (>40) during pre-encryption wipe.
8	registry_activity_count	Sum of Event IDs 12, 13	Spikes when altering security settings.
9	network_activity_count	Sum of Event ID 3	Outbound socket creation count.
10-17	event_1..26_count	Individual Event ID frequencies	Granular event distribution counters.

Module 4 (Continued): Vectorization Normalization Formula

To ensure that high-magnitude integer counts (e.g. ``file_activity_count = 120``) do not overwhelm binary flags (e.g. ``suspicious_path_count = 1``), all 17 features undergo standard scaling prior to inference:

$$x' = \frac{x - \mu}{\sigma}$$
where μ is the empirical feature mean and σ is the standard deviation fitted on 20,402 benchmark windows.

Module 5: Step 3 — Tier 1 Baseline Anomaly Screening

BRDS-PEC employs a **2-Tier Machine Learning Architecture** to balance sub-millisecond screening speed with deep sequence classification accuracy:

- **Tier 1 Role:** Screen tens of thousands of routine background OS events (e.g. chrome.exe opening tabs or explorer.exe rendering icons) with sub-millisecond latency (<1 ms per window).
- **Tier 1 Algorithms:** Standard Scaling + Logistic Regression pipeline trained with `class_weight='balanced'` plus Isolation Forest anomaly detection.
- **Decision Threshold:** If a 5-second feature vector evaluates to a Risk Score ≥ 0.50 , it is flagged as an anomaly and passed immediately to Tier 2 for deep sequence evaluation.
- **Tier 1 Benchmark Metrics:** Achieves **99.51% F1 Score**, **99.53% Precision**, **99.48% Recall**, and **0.22% False Positive Rate** on 20,402 evaluated windows.

Module 6: Step 4 — Tier 2 PyTorch Bidirectional LSTM Sequence Classifier

Single-window classifiers can be tricked by obfuscated ransomware that pauses between actions. Tier 2 evaluates **30-step chronological sequences** (150 seconds of process history) using a 2-Layer Bidirectional LSTM (`ml_engine/lstm/model.py`).

```
INPUT TENSOR: [Batch Size: 32, Sequence Length: 30, Features: 17]
↓
LAYER 1: Linear Input Projection (17 -> 64 dimensions) | Params: 1,152
↓
LAYER 2: 2-Layer Stacked Bidirectional LSTM (Hidden size: 64, BiLSTM Output: 128) | Params: 165,888
↓
LAYER 3: Concatenated Mean Pooling (128) + Max Pooling (128) -> Representation Vector (256)
↓
LAYER 4: Fully Connected FC Output Layer (256 -> 1) + Sigmoid Activation | Params: 257
↓
FINAL RISK PROBABILITY: Output in range [0.0, 1.0] | TOTAL PARAMS: 167,297
```

Module 6 (Continued): Parameter Formula Derivation & RCE Hardening

Exact Parameter Count Formula:

An LSTM cell calculates weights using: $4 \times ((\text{inputs} + \text{hidden}) \times \text{hidden} + \text{hidden})$.

For Layer 1 ($D_{in}=64$, $H=64$, Bidirectional $\times 2$): $2 \times [4 \times ((64 + 64) \times 64 + 64)] = 66,048$ params.

For Layer 2 ($D_{in}=128$, $H=64$, Bidirectional $\times 2$): $2 \times [4 \times ((128 + 64) \times 64 + 64)] = 98,816$ params.

Input Projection: $(17 \times 64) + 64 = 1,152$ params.

Output FC: $(256 \times 1) + 1 = 257$ params.

Total Trainable Model Parameters = 167,297

Model Deserialization Hardening (`ml_engine/lstm/infer.py`):

- **Cryptographic Checksum Verification:** Before loading weights, `infer.py` verifies `lstm_model.pth` against `lstm_model.sha256` digest.
- **Weights-Only Safe Loading:** Model checkpoint is loaded using `torch.load(weights_only=True)`, blocking arbitrary Python pickle Remote Code Execution (RCE).

Module 7: Step 5 — Cryptographic HMAC-SHA256 Alert Signing & Arm Tokens

In enterprise environments, sophisticated malware might attempt to inject fake security alerts or tamper with alert files on disk to trigger false host isolations. BRDS-PEC enforces strict cryptographic protection via `containment/alert_integrity.py`:

- **HMAC-SHA256 Alert Signing:** Alerts are serialized to JSON and signed using an environment secret key (``$env:BRDS_ALERT_HMAC_KEY``).
- **Single-Use `.arm_token`` Authorization:** When a high-risk alert (`Risk >= 0.85`) is generated, the system creates a single-use `.arm_token`` file in ``data/processed/``. The containment daemon consumes and deletes this token before executing any host isolation script.

Module 8: Step 6 — Host Containment System & Protected Process Denylist

When an alert with Risk ≥ 0.85 and a valid `.arm_token` is verified by `containment/trigger_daemon.py`, it triggers automated host containment:

1. **Host Network Adapter Isolation (`containment/ContainHost.ps1`):** Disables active network adapters (`Disable-NetAdapter`), flushes ARP/DNS caches, and applies Windows Firewall block rules to prevent SMB/RDP lateral spread.
2. **Bottom-Up Process Tree Collapse (`containment/kill_process_tree.ps1`):** Recursively queries child processes and executes `taskkill /F /T` on the ransomware PID tree.
3. **OS Protected Process Denylist (BSOD Prevention):** Core Windows system processes (`lsass`, `csrss`, `smss`, `wininit`, `services`, `svchost`, `explorer`, `dwm`) are explicitly protected by `$PROTECTED_PROCESSES` so the OS does NOT crash.

Module 9: Step 7 — Explainable AI (XAI) & PyTorch Autograd Gradients

SOC analysts cannot trust black-box AI predictions. BRDS-PEC provides full mathematical explainability (`ml_engine/xai/shap_explainer.py`):

- **PyTorch Autograd Integrated Gradients:** Computes exact numerical feature attributions ($|\nabla_x y * x|$) showing which Sysmon features drove the high risk score.
- **Downloadable PDF Reports:** Analysts can click `SHAP Analysis` on any incident card and download an official PDF report generated by ReportLab (`backend/routes/xai_routes.py`).

Module 10: Dataset Governance & Generalization Proof

BRDS-PEC was evaluated on **20,402 windowed records** across 5 datasets:

Dataset Name	Total Windows	Role & Threat Scenario Covered
SILRAD-1.0	17,617 Benign	Genuine Windows 11 baseline user activity (Chrome, Explorer, Defender).
Splunk ATT&CK;	2,785 Attack	Authentic Sysmon logs for WannaCry, LockBit, Ryuk, BlackBasta, Sodinokibi.
CSU Ransomware	Secondary Validation	Extracted via `extract_goodware.py` for cross-dataset validation.
MLRAN Dataset	Secondary Validation	Cuckoo sandbox API call logs processed for feature robustness auditing.
RansomSet	Secondary Validation	Multi-class correlation reference for validating SHAP attributions.

Leave-One-Scenario-Out (LOSO) Cross-Validation Results:

To prove zero-day generalization, entire ransomware families were held out from training:

- **Held-Out LockBit (`atomic\_red\_team`):** 94.86% F1 Score
- **Held-Out SamSam (`sam\_sam\_note`):** 99.71% F1 Score
- **Held-Out General Ransomware (`ransomware\_notes`):** 99.74% F1 Score
- **AVERAGE HELD-OUT UNSEEN ATTACK F1 SCORE: 93.14%**

Module 11: Top 5 Core Presentation Code Blocks

1. PyTorch LSTM Architecture (`ml\_engine/lstm/model.py`):

```
class LSTMClassifier(nn.Module): ... lstm_out.mean(dim=1) + lstm_out.max(dim=1).values ... fc(256->1)
```

2. RCE Hardening & Hash Check (`ml\_engine/lstm/infer.py`):

```
torch.load(weights_only=True) & hmac.compare_digest(expected_hash, actual_hash)
```

3. HMAC-SHA256 Alert Integrity (`containment/alert\_integrity.py`):

```
hmac.new(key, payload.encode(), hashlib.sha256).hexdigest()
```

4. OS Process Denylist Protection (`containment/kill\_process\_tree.ps1`):

```
$PROTECTED_PROCESSES = @('lsass', 'csrss', 'svchost', 'explorer', 'services')
```

5. PyTorch Autograd XAI (`ml\_engine/xai/shap\_explainer.py`):

```
output.backward(); grad_x_input = np.abs(input_tensor.grad * features_scaled).mean(dim=0)
```

Module 12: Master Reviewer Q&A; Cheat Sheet & Live VM Demo Playbook

Q1: How does BRDS-PEC differ from traditional Antivirus or EDR?

Answer: Antivirus relies on static file hashes easily bypassed by re-packing. Commercial EDRs often trigger alerts after files are encrypted. BRDS-PEC evaluates sliding 5-second Sysmon behavioral windows using PyTorch LSTM to halt execution in the pre-encryption phase.

Q2: Why use a 2-Tier Machine Learning architecture?

Answer: Tier 1 (Logistic Regression + Isolation Forest) rapidly screens tens of thousands of background events with sub-millisecond latency. Tier 2 (PyTorch BiLSTM) evaluates 30-step process history only for flagged anomalies, balancing speed and deep sequence accuracy.

Q3: How do you prove the model detects unseen ransomware families?

Answer: We performed Leave-One-Scenario-Out Cross-Validation (``scenario_holdout_eval()``). The model achieved 93.14% Average F1 Score on ransomware attack scenarios completely held out from training.

Q4: How do you prevent auto-containment from causing Blue Screen of Death (BSOD)?

Answer: ``kill_process_tree.ps1`` checks a ``$PROTECTED_PROCESSES`` denylist (``lsass``, ``csrss``, ``svchost``, ``explorer``, ``services``) before process tree termination.

Q5: What stops malware from forging fake security alerts?

Answer: ``containment/alert_integrity.py`` signs all alert containers using HMAC-SHA256 and issues single-use ``arm_token`` authorization files.

Q6: How to run a live demonstration inside a VM?

Answer: Launch Terminal 1 (``python backend/app.py``), Terminal 2 (``python pipeline/watchdog.py``), Terminal 3 (``$env:BRDS_DRY_RUN='0'; python containment/trigger_daemon.py``), then execute the PowerShell ransomware simulation script in Terminal 4 to see live threat spikes and auto-containment!